
Semana 23 - Aula 1

Tópico Principal da Aula: Frameworks Back-end e Integração com Serviços Externos

Subtítulo/Tema Específico: Desenvolvimento Avançado com Express.js

Código da aula: [SIS]ANO2C2B4S23A1

Objetivos da Aula:

- Desenvolver aplicações *back-end* mais robustas e eficientes utilizando o *framework* Express.js.
- Dominar a configuração de **rotas**, a aplicação de **middlewares** e a **gestão de erros** de forma profissional em ambientes Node.js.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet para práticas de código.

Exposição do Conteúdo:

Referência do Slide: Slide 06 - Configuração de Rotas Avançadas

- **Definição:** Express.js é um framework para Node.js usado para criar aplicações web e APIs de forma simples e rápida.
Ele facilita o tratamento de rotas, requisições HTTP, respostas, e o uso de middlewares (funções intermediárias que processam dados entre o pedido do cliente e a resposta do servidor).

```
javascript 📄 Copiar código

// Importa o módulo express
const express = require('express');

// Cria uma aplicação express
const app = express();

// Define uma rota GET na raiz "/"
app.get('/', (req, res) => {
  res.send('Bem-vindo ao meu servidor com Express.js!');
});

// Define a porta onde o servidor vai rodar
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

O que acontece nesse exemplo:

- O Express é importado e uma aplicação é criada.

- É configurada uma rota principal (/) que responde com uma mensagem.
- O servidor é iniciado na porta 3000.
- Ao acessar <http://localhost:3000> no navegador, aparece a mensagem “Bem-vindo ao meu servidor com Express.js!”.
- **As Rotas** em Express.js definem como o aplicativo responde a uma requisição do cliente em um *endpoint* específico (URI) e com um método HTTP particular (GET, POST, PUT, DELETE). A configuração avançada envolve o uso de objetos `express.Router()` para modularizar o código e organizá-lo em arquivos separados por funcionalidade ou recurso.
- **Aprofundamento/Complemento (se necessário):** A modularização de rotas é fundamental em aplicações de grande porte, pois evita o crescimento excessivo do arquivo principal (`app.js` ou `server.js`). O objeto `Router` funciona como um *mini-aplicativo*, isolando um conjunto de rotas e *middlewares* relacionados.
- **Exemplo Prático:** Em um sistema de e-commerce, todas as rotas relacionadas a pedidos (`/pedidos`) podem ser agrupadas em um arquivo `pedidos.js` usando `express.Router()`. No arquivo principal, o roteador é montado: `app.use('/pedidos', pedidosRouter)`. Isso garante que todas as rotas dentro de `pedidosRouter` (como `/` para listar ou `/:id` para detalhar) já estejam prefixadas com `/pedidos`.
 - **Link de vídeo 1:** [Express.js Routing and Router](#) (A partir de 30:43, fala sobre `app.use()` e rotas - Adaptado da busca sobre Middleware).
 - **Link de vídeo 2:** [Introdução ao Express/Node - Roteamento \(MDN\)](#)

Referência do Slide: Slide 09 - Middlewares e Sua Aplicação

- **Definição:** Um **Middleware** (função intermediária) é uma função que tem acesso aos objetos de requisição (`req`), resposta (`res`) e à próxima função de *middleware* no ciclo de requisição-resposta (`next`). Eles são usados para executar código, modificar objetos de requisição e resposta, finalizar o ciclo ou chamar o próximo *middleware*.
- **Aprofundamento/Complemento (se necessário):** Os *middlewares* podem ser de nível de aplicação (`app.use()`), de nível de roteador, de tratamento de erros ou *built-in* (nativos do Express). Eles são essenciais para tarefas como autenticação, log, validação de dados e compressão. A ordem em que são declarados é crucial, pois define a ordem de execução.
- **Exemplo Prático:** Criar um *middleware* de log que registra o horário, o método HTTP e o *endpoint* de cada requisição antes de passar para a rota principal.
- JavaScript

```
const logger = (req, res, next) => {
  console.log(`[LOG] ${new Date().toISOString()} - ${req.method} ${req.originalUrl}`);
  next(); // Passa o controle para o próximo middleware ou rota
};
app.use(logger);
```

- - **Link de vídeo 1:** [What is Middleware in Express JS?](#) (Vídeo em inglês, focado em explicar o conceito e tipos de middleware).
 - **Link de vídeo 2:** [Usando middlewares do Express \(MDN\)](#)

Referência do Slide: Slide 12 - Gestão Eficaz de Erros

- **Definição:** Em Express.js, o **tratamento de erros** deve ser feito através de funções de *middleware* especiais que aceitam quatro argumentos: (err, req, res, next). Uma gestão eficaz de erros não só melhora a experiência do usuário, como também facilita a manutenção e depuração do código. O Express possui um *handler* de erro padrão, mas é recomendado criar um *customizado*.
- **Aprofundamento/Complemento (se necessário):** Para que erros em código **assíncrono** (como Promises sem try/catch ou async/await) sejam capturados, é necessário envolvê-los em try/catch ou usar bibliotecas como express-async-errors. O *middleware* de erro deve ser sempre o último app.use() a ser declarado.
- **Exemplo Prático:** Um *middleware* que detecta o erro e o transforma em uma resposta JSON padronizada (Ex: {"status": 500, "message": "Erro Interno do Servidor"}).
- JavaScript

```
app.use((err, req, res, next) => {  
  console.error(err.stack); // Loga o erro para o desenvolvedor  
  const statusCode = err.status || 500;  
  res.status(statusCode).send({  
    status: statusCode,  
    message: err.message || 'Erro interno do servidor'  
  });  
});
```

- - **Link de vídeo 1:** [Tratamento de erros no express em rotas com async/await - YouTube](#)
 - **Link de vídeo 2:** [Express.js Error Handling: A Deep Dive into Error Middleware](#)

Semana 23 - Aula 2

Tópico Principal da Aula: Frameworks Back-end e Integração com Serviços Externos

Subtítulo/Tema Específico: Utilização de Django: Modelos, Templates e Segurança

Código da aula: [SIS]ANO2C2B4S23A2

Objetivos da Aula:

- Utilizar o *framework* Django para desenvolvimento rápido, focando na arquitetura **MVT (Model-View-Template)**.
- Implementar **modelos** para gestão de dados e criar **templates** para renderização dinâmica.
- Aplicar as ferramentas de **segurança** nativas do Django, como CSRF e sanitização, para construir aplicações web seguras.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet para configuração e prática de projetos Django.

Exposição do Conteúdo:

Referência do Slide: Slide 06 - Modelos e a Camada de Dados

- **Definição:** Django é um framework web em Python que permite desenvolver sites e aplicações web completas de forma rápida, segura e escalável. Ele segue o padrão MVC (Model–View–Controller) — no Django chamado de MVT (Model–View–Template).

Origem do nome

O nome “**Django**” vem do guitarrista **Django Reinhardt**, um músico de jazz famoso por sua habilidade e improvisação.

Os criadores do framework (Adrian Holovaty e Simon Willison) eram fãs de jazz e escolheram o nome em homenagem a ele.

Principais recursos do Django

- ORM (Mapeamento Objeto-Relacional) para interagir com bancos de dados sem SQL puro
- Painel administrativo pronto automaticamente
- Sistema de rotas e templates integrado
- Módulos de autenticação e segurança embutidos
- Ideal para aplicações robustas e profissionais (ex: Instagram começou com Django)

Exemplo simples

Criando uma view no Django

```
python

# arquivo: views.py
from django.http import HttpResponse

def home(request):
    return HttpResponse("Bem-vindo ao meu site feito com Django!")
```

 Copiar código

Definindo a rota correspondente

```
python

# arquivo: urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home), # Rota principal
]
```

 Copiar código

🌱 O que acontece:

- Quando o usuário acessa `http://localhost:8000/`, o Django chama a função `home()`.
- Essa função retorna uma resposta HTTP com o texto **"Bem-vindo ao meu site feito com Django!"**.
- Todo o processo (requisição → processamento → resposta) é gerenciado automaticamente pelo framework.
- **Os Modelos no Django definem a estrutura dos dados da aplicação, representando tabelas em um banco de dados.** Eles são a essência da camada *Model* no padrão MVT (Model-View-Template) do Django.
- **Aprofundamento/Complemento (se necessário):** O Django utiliza um **ORM (Object-Relational Mapper)**, que permite interagir com o banco de dados usando código Python, em vez de SQL. A **migração** é o processo de aplicar as alterações feitas nos modelos ao esquema do banco de dados (`makemigrations` e `migrate`).
- **Exemplo Prático:** Criação de um modelo simples de **Postagem** em um blog.
- Python

```
from django.db import models
```

```
class Postagem(models.Model):
```

```
    titulo = models.CharField(max_length=200)
```

```
    conteudo = models.TextField()
```

```
    data_publicacao = models.DateTimeField(auto_now_add=True)
```

```
    autor = models.ForeignKey('auth.User', on_delete=models.CASCADE)
```

```
    def __str__(self):
```

```
        return self.titulo
```

- - **Link de vídeo 1:** [How to Define a Custom Django Model](#) (Parte 11 de um tutorial de Django).
 - **Link de vídeo 2:** [Introdução ao Django: Modelos e Templates](#)

Referência do Slide: Slide 09 - Templates e Renderização Dinâmica

- **Definição: Templates** são arquivos de texto (geralmente HTML) que definem a estrutura da interface do usuário. Eles contêm *tags* e *variáveis* específicas do **Django Template Language (DTL)**, permitindo a **renderização dinâmica** de dados fornecidos pelas *Views*.
- **Aprofundamento/Complemento (se necessário):** A DTL permite a inclusão de lógica simples (como *loops* e condicionais) nos templates. É prática recomendada o uso de **templates base** (`base.html`) com a *tag* `{% block %}` para definir a estrutura principal e permitir que outros templates a herdem com a *tag* `{% extends %}`.
- **Exemplo Prático:** Um *template* que exibe uma lista de postagens usando um *loop*.
- HTML

```
{% extends "base.html" %}

{% block content %}
<h1>Últimas Postagens</h1>
{% for postagem in postagens %}
  <h2>{{ postagem.titulo }}</h2>
  <p>{{ postagem.conteudo|truncatechars:100 }}</p>
{% empty %}
  <p>Nenhuma postagem encontrada.</p>
{% endfor %}
{% endblock %}
```

- - **Link de vídeo 1:** [How to Use Django HTML Templates](#) (Parte 3 de um tutorial de Django).
 - **Link de vídeo 2:** [Building the Django Base HTML Template](#) (Parte 13 de um tutorial de Django).

Referência do Slide: Slide 12 - Segurança: CSRF, XSS e Sanitização

- **Definição:** O Django é conhecido por ter mecanismos de **segurança embutidos** que protegem a aplicação contra ataques comuns. O **CSRF (Cross-Site Request Forgery)** é mitigado pelo *middleware* e *tag* `{% csrf_token %}` em formulários POST. A **Sanitização** de dados, para prevenção de **XSS (Cross-Site Scripting)**, é feita automaticamente pela DTL ao escapar HTML por padrão.
- **Aprofundamento/Complemento (se necessário):** O Django também oferece proteção contra **SQL Injection** através do seu ORM. A sanitização padrão pode ser desabilitada usando a *tag* `|safe`, mas isso deve ser feito apenas quando for **absolutamente necessário** e se a origem do conteúdo for confiável. Outra proteção importante é a necessidade de configurar o `ALLOWED_HOSTS` para evitar ataques de *header poisoning*.
- **Exemplo Prático:** Uso da *tag* de segurança CSRF em um formulário de login.
- HTML

```
<form method="post">
  {% csrf_token %}
  <input type="submit" value="Enviar">
</form>
```

- - **Link de vídeo 1:** [A Beginners Guide to Security Exploits in Action](#) (Discussão sobre vulnerabilidades e patches no Django).
 - **Link de vídeo 2:** [Segurança de aplicações web Django \(MDN\)](#)

Semana 23 - Aula 3

Tópico Principal da Aula: Frameworks Back-end e Integração com Serviços Externos

Subtítulo/Tema Específico: Integração Contínua com Jenkins

Código da aula: [SIS]ANO2C2B4S23A3

Objetivos da Aula:

- Implementar o conceito de **Integração Contínua (CI)** utilizando a ferramenta Jenkins.
- Configurar **pipelines automatizadas** que englobam desde a compilação do código até o *deployment*.
- Compreender a importância da **automação de testes** e do **deployment contínuo** no ciclo de vida do software.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática para instalação e configuração do Jenkins em ambiente virtual ou *container*.

Exposição do Conteúdo:

Referência do Slide: Slide 08 - Configuração de Pipelines

Definição: Jenkins é uma das ferramentas mais populares para implementar Integração Contínua. Ele é um **servidor de automação open-source**, desenvolvido em Java, que permite criar **pipelines** (fluxos de automação) para:

- Compilar o projeto automaticamente
- Rodar testes
- Fazer deploy (publicação)
- Gerar relatórios e notificações

Como funciona na prática

1. O desenvolvedor faz alterações no código e envia (push) para o repositório.
 2. O **Jenkins detecta a nova versão** do código (via webhook do GitHub, por exemplo).
 3. Ele **executa o pipeline configurado**, que pode ter etapas como:
 - Baixar o código
 - Instalar dependências
 - Executar testes
 - Gerar build
 - Fazer deploy em ambiente de teste ou produção
 4. O Jenkins informa se o build foi **"sucesso" ou "falhou"**.
- Uma **Pipeline Jenkins** é uma suíte de *plugins* que suporta a implementação e integração de *pipelines* de entrega contínua (CD) em Jenkins. Ela define um fluxo de trabalho automatizado que gerencia todas as etapas, desde a compilação do código até o *deployment* em produção.

- **Aprofundamento/Complemento (se necessário):** As *pipelines* podem ser definidas de forma **Declarativa** (sintaxe mais simples e estruturada, preferida) ou **Scripted** (baseada em Groovy, mais flexível). O uso do **Jenkinsfile** (um arquivo que define a *pipeline* e é versionado junto com o código-fonte) garante rastreabilidade e consistência, seguindo a prática de "Pipeline-as-Code".
- **Exemplo Prático:** Configuração de uma *pipeline* simples com as etapas (*stages*) de **Checkout**, **Build** e **Testes** para uma aplicação Node.js.
- Groovy

```
// Jenkinsfile Declarativo
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git url: 'seu_repositorio_git' }
    }
    stage('Build') {
      steps { sh 'npm install' }
    }
    stage('Test') {
      steps { sh 'npm test' }
    }
  }
}
```

- - **Link de vídeo 1:** [O básico de Pipelines CI/CD com Jenkins - YouTube](#)
 - **Link de vídeo 2:** [How to set up continuous deployment pipelines with Jenkins](#)

Referência do Slide: Slide 11 - Automação de Testes

- **Definição:** A **Automação de Testes** consiste em utilizar ferramentas e *scripts* para executar testes automaticamente durante o ciclo de desenvolvimento. Em um ambiente de CI, é fundamental para **validar rapidamente as alterações** e manter a integridade do aplicativo.
- **Aprofundamento/Complemento (se necessário):** A automação de testes deve incluir **testes unitários** (para validar funções isoladas), **testes de integração** (para verificar a comunicação entre componentes) e, idealmente, **testes end-to-end**. A integração de ferramentas como JUnit (Java), Playwright/Cypress (JavaScript) ou Pytest (Python) no *stage* de Testes da *pipeline* garante que apenas códigos validados prossigam.
- **Exemplo Prático:** Configurar um *stage* de testes no Jenkins para executar testes em um **ambiente isolado** (como um *container* Docker) para garantir que os testes sejam padronizados e confiáveis.
 - **Link de vídeo 1:** [CI \(Teste automatizado\) com Jenkins, Github e Python - YouTube](#)
 - **Link de vídeo 2:** [Crie um Pipeline de Testes Contínuos com Jenkins](#)

Referência do Slide: Slide 14 - Deployment Contínuo e Boas Práticas

- **Definição:** O **Deployment Contínuo (CD)** é uma prática de engenharia de software na qual qualquer alteração de código que passou em todos os testes e etapas da *pipeline* é **automaticamente** lançada em produção, sem intervenção humana.
- **Aprofundamento/Complemento (se necessário):** Boas práticas de CD incluem o **monitoramento contínuo** (integrar com ferramentas de observabilidade para detectar problemas em produção rapidamente), a **automação completa** (minimizar a intervenção humana) e a execução de **testes pós-deployment** (validações finais na versão em produção). A **gestão de rollback** é crucial para reverter rapidamente a versão em caso de falha no *deployment*.
- **Exemplo Prático:** Adicionar um *stage* final à *pipeline* do Jenkins chamado **Deploy** que, após o sucesso dos testes, executa um *script* SSH ou um comando de ferramenta de *container* (ex: Docker Compose, Kubernetes) para atualizar o servidor de produção.
 - **Link de vídeo 1:** [How to set up continuous deployment pipelines with Jenkins](#)
 - **Link de vídeo 2:** [Pipeline CI/CD, tipos de testes e principais ferramentas - YouTube](#)
 -